COSC419/519:DEEP LEARNING

Transformers

Dr. Mohamed Shehata

Computer Science

University of British Columbia

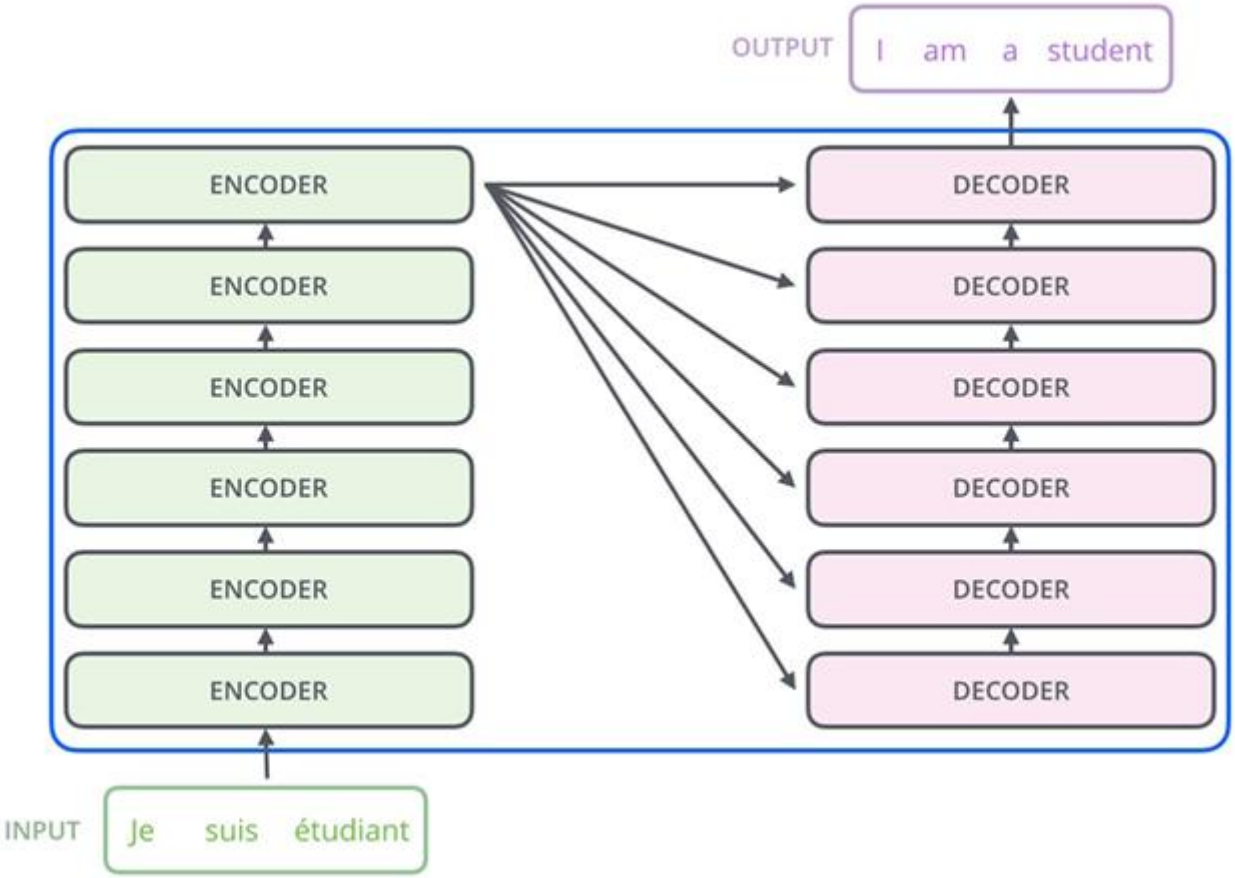
UBC OKANAGAN

Still the same paper:
Vaswani, A. et al. "Attention is all you need“ Advances in Neural Information Processing Systems (NIPS 2017).

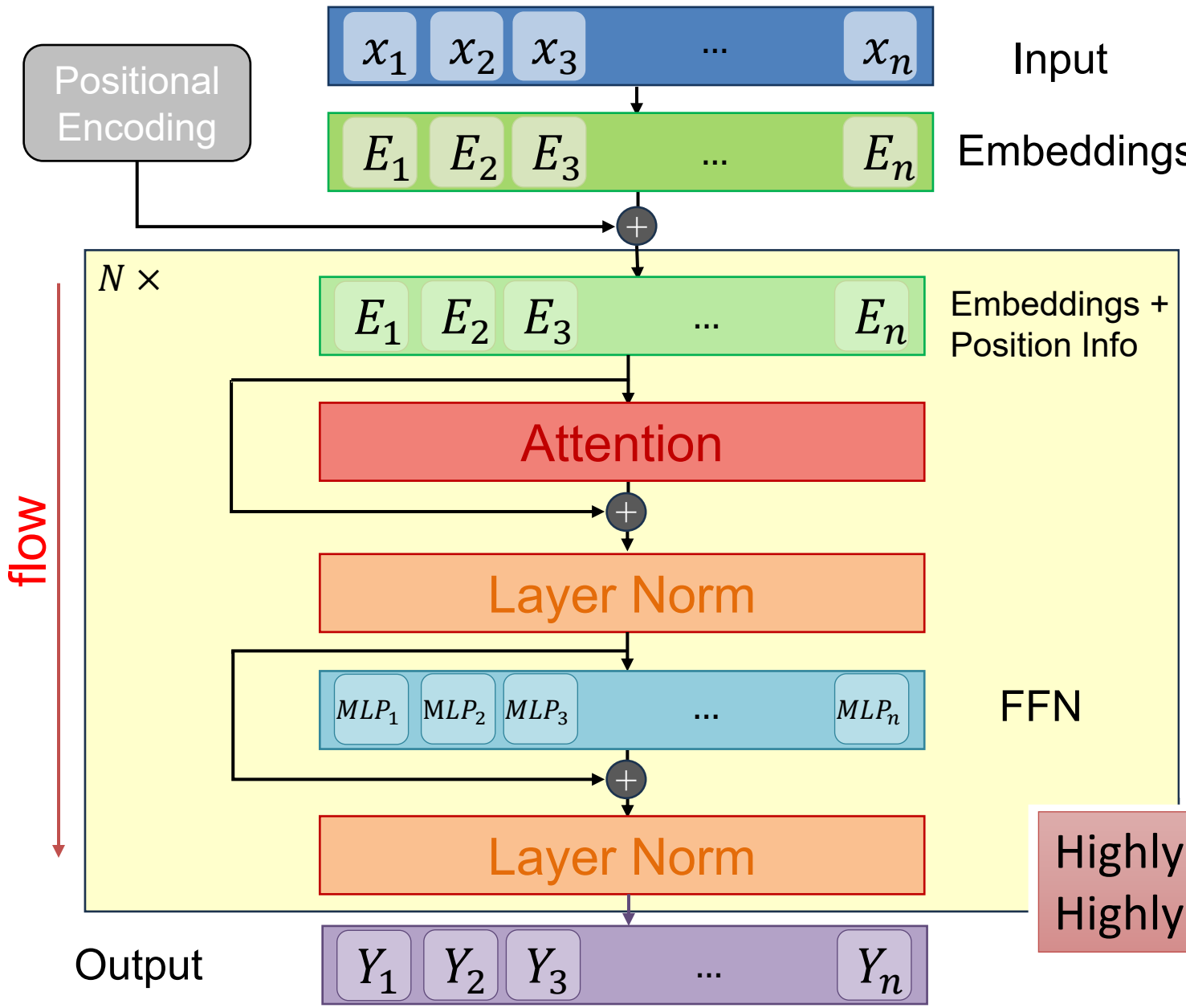
The Transformers

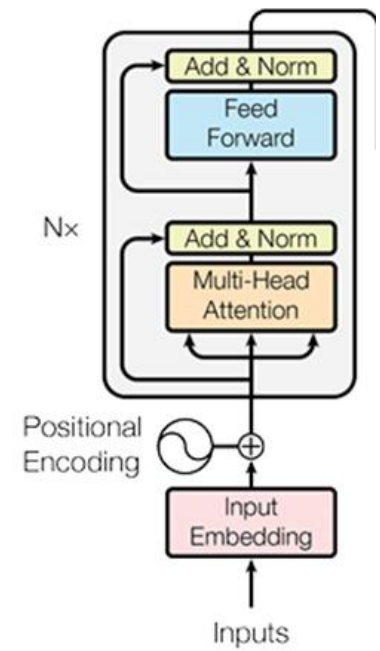
- A transformer is a stack of transformer block:
  - The Encoder consists of N identical encoder blocks processing the input embeddings into context-aware representations
  - The Decoder consists of N identical decoder blocks, generating output sequences based on the encoder output and previously generated tokens

Example of Transformer Main Body: Encoder-Decoder



The Transformer Encoder Block

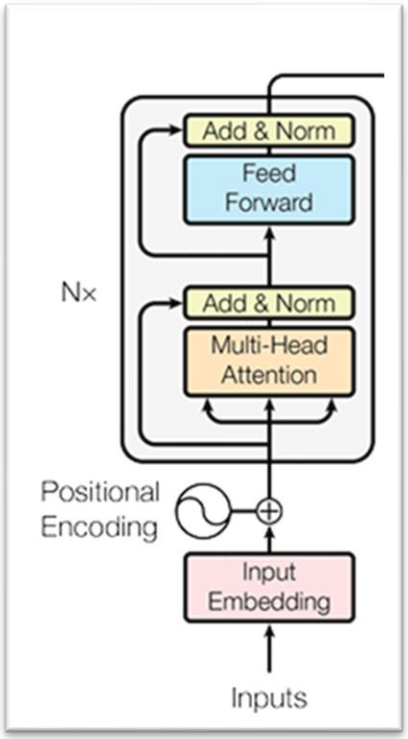


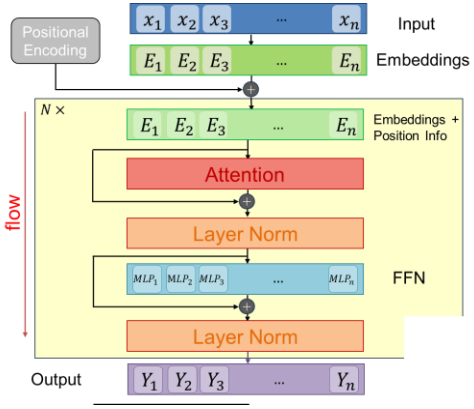


Highly Parallelizable
Highly Generalizable

Transformers

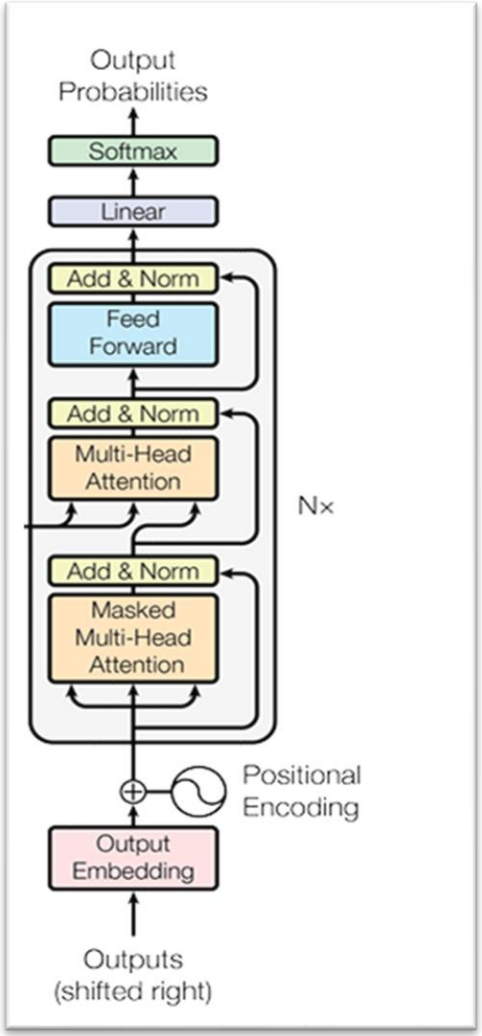
- Transformer encoder blocks:
  - Input: set of vectors X
  - Output: set of vectors Y
- A Transformer Encoder is just a **series of Transformer encoder blocks** repeated N times.
- In the original Vaswani et. al paper:
  - 6 blocks
  - $d_{model} = 512$
  - 6 heads





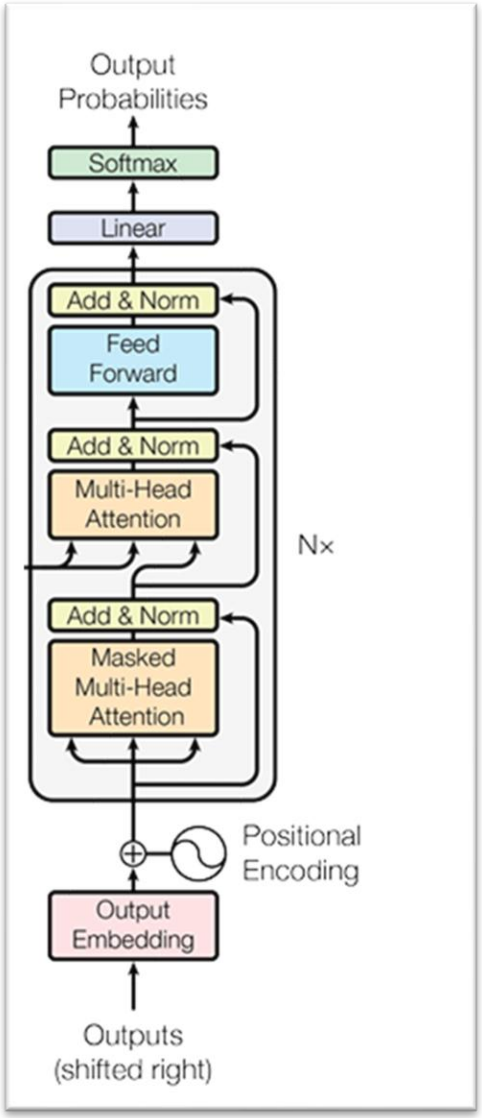
Transformer Decoder Block

- Most parts of a decoder block are the same as encoder blocks, with some slight modifications and additions, for which we will go into details later.
  - The positional encoding and token embeddings are exactly the same.
  - The Add & Norm sections are exactly the same.
  - The first multi-head attention (MHA) performs self-attention as before, but with an additional **masking** step.
  - The second multi-head attention performs **cross-attention**, including the outputs of the final encoder block and the previous MHA.
  - The Feed Forward (MLP) stays the same.
  - There is an additional **linear projection** after the output of the final decoder block to convert outputs from $(n \times d_{model})$ to $(n \times \text{vocab-size})$
    - Performing column-wise softmax on this $(n \times \text{vocab-size})$ matrix provides a probability distribution over the entire vocabulary for each column (i.e. each position in the sequence)



Transformer Decoder Block

- The input to the decoder block is just the sequence of tokens shifted right by one position, and the **final word of the sequence is left out as the target**.
- Consider the sentence "The cats like chasing mice" as an example:
  - Decoder Input = [<start>, "The", "cats", "like", "chasing""]
  - The right-shift ensures that the input only contains tokens **before** the target token. The model cannot peak into future tokens
  - <start> is a special token, among others such as <unk> (for tokens outside the vocabulary) and <pad> (for padding a sequence to a fixed length)
  - Using these inputs and passing them through a decoder block, we get an output sequence of the same length (n) but instead of each vector being size d_{model}, they are size $|V|$ (since each column is a probability distribution over the vocabulary)
  - We go through iterations to train the transformer



Training a Decoder

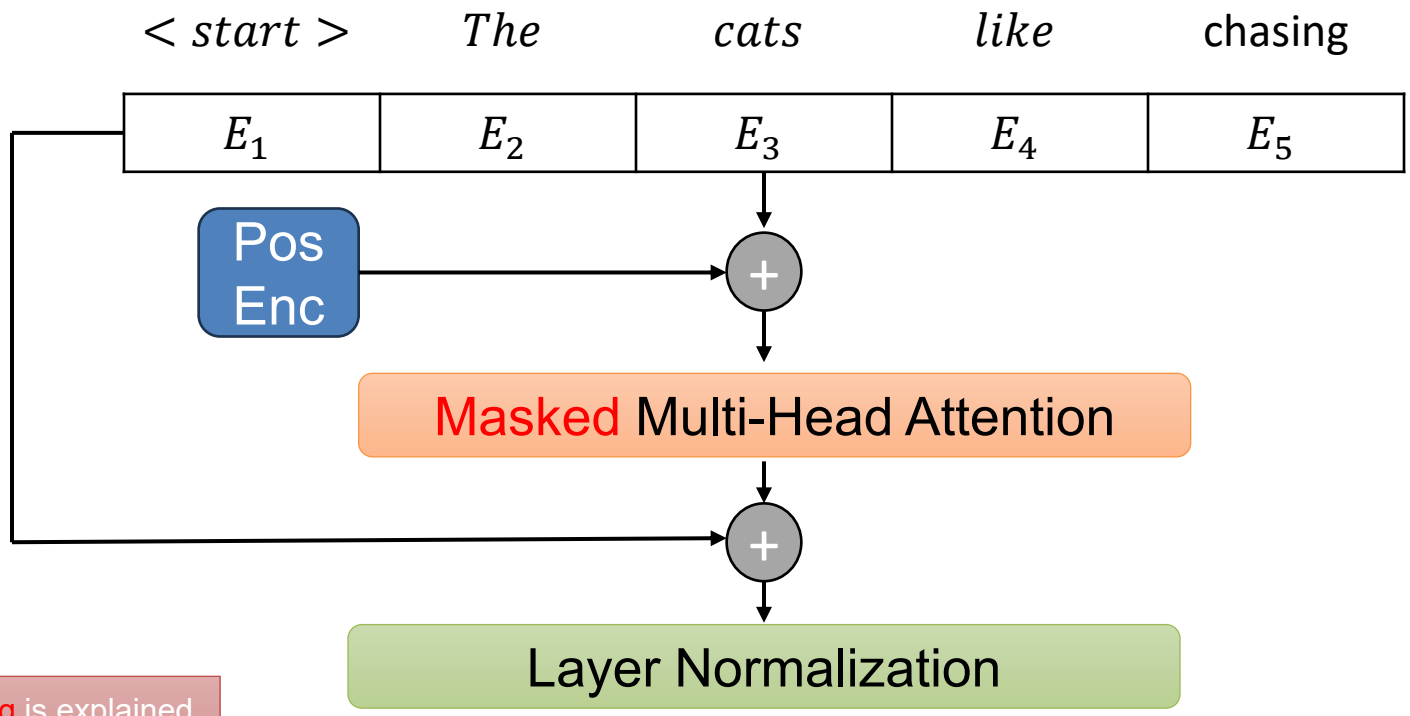
- We can use an iterative approach to create multiple training examples from the same sequence (not really iterative but let us build the concept first).
- For example: In Autoregressive Language Modeling (LM)

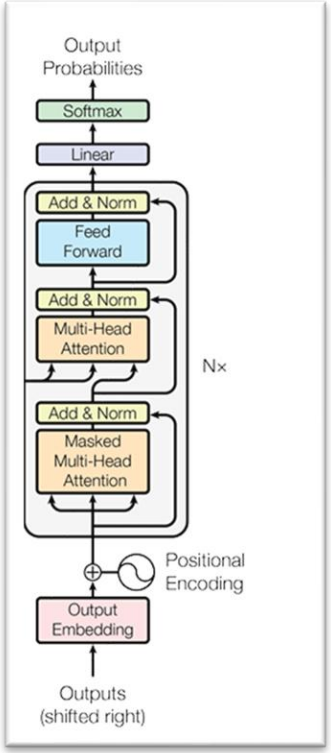
Iteration	Decoder Input	Target
1	<start>	The
2	<start>, The	cats
3	<start>, The, cats	Like
4	<start>, The, cats, like	chasing
5	<start>, The, cats, like, chasing	mice

- And in each iteration, the model compares the prediction with the expected output and uses a loss function to optimize the parameters.
  - In the Vaswani et. al paper, they used Cross Entropy loss and an Adam optimizer (with learning rate warm up)

First Multi-Head Attention Step

- The first multi-head self attention remains mostly the same as what we saw earlier in the encoder blocks.
  - The input is n vectors of size d_{model}
  - Attention scores are computed, added to embeddings, followed by layer normalization
  - The output contains "modified" embeddings for each token.





Masking is explained on the next slide

Masking

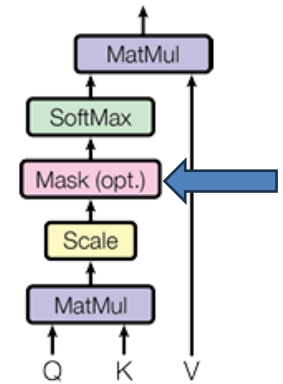
- Masking is used **only in the decoder**, and **only in the first MHA** step. It prevents the model from "cheating" by looking at future tokens for computing the similarities of the current token.

e.g.: the similarities for "like" and "chasing" should be masked when calculating a score for "cats"

Query vectors →

Key vectors ↓

	The	cats	Like	chasing
The				
cats	X			
like	X	X		
chasing	X	X	X	



- It is done by setting all the similarities for tokens after the current token to $-\infty$ **before** the SoftMax.
  - After SoftMax, these values will be set to 0.
  - The query Q focuses on the current token, and the key vectors come from only the previous tokens.

$-\infty$			
$-\infty$	$-\infty$		
$-\infty$	$-\infty$	$-\infty$	

column-wise softmax →

0			
0	0		
0	0	0	

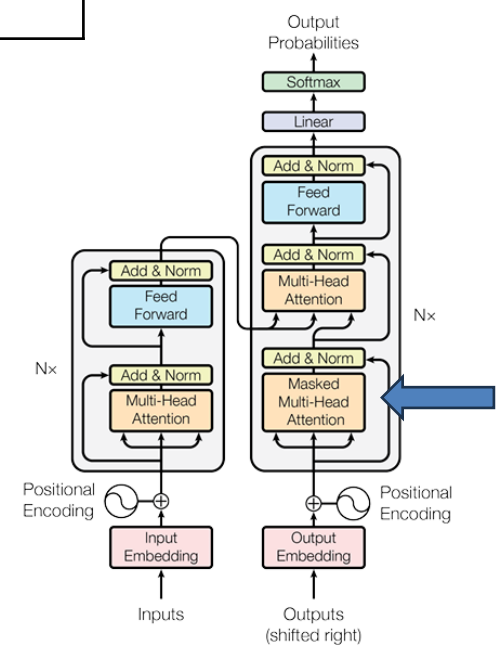


Figure 1: The Transformer - model architecture.

Second Multi-Head Attention Step

- This step is also known as the "encoder-decoder" step. It integrates contextual information from the decoder with the partially generated sequence in the decoder.
- It does so by using the output $Y_{enc} = [Y_1, Y_2 \ldots Y_n]$ of the **encoder** layer for the **keys and values** of the attention step.
  - In particular, the output of the encoder Y_{enc} is multiplied by the learned matrices W_K and W_V to get K and V:
  - So $K = Y_{enc} \cdot W_{K_dec}$ and $V = Y_{enc} \cdot W_{V_dec}$
    - Y_{enc}: $(n_{enc} \times d_{model})$, and W_{K_dec}: $(d_{model} \times d_k)$ so K: $(n_{enc} \times d_k)$
    - Y_{enc}: $(n_{enc} \times d_{model})$, and W_{V_dec}: $(d_{model} \times d_v)$ so V: $(n_{enc} \times d_v)$
- The query Q comes from the output of the previous multi-head attention step in the decoder.
  - $Q = E \cdot W_{Q_dec}$
    - E: $(n_{dec} \times d_{model})$, and W_Q: $(d_{model} \times d_k)$ so Q: $(n_{dec} \times d_k)$

Second Multi-Head Attention Step

- Notice that query matrix Q with shape $(n_{dec} \times d_k)$ is of different length (n) than the key matrix K with shape $(n_{enc} \times d_k)$. Why is this?
  - We saw earlier that the input to the decoder does not have to be the entire sequence; we could have used only part of the 5-word sentence, say the first 3 words. Therefore, the output of our first decoder step would be a sequence of $n_{dec} = 3$.
- So what does our attention matrix look like with this Q and K?

$$S = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}^{n_{dec} \times n_{enc}}$$

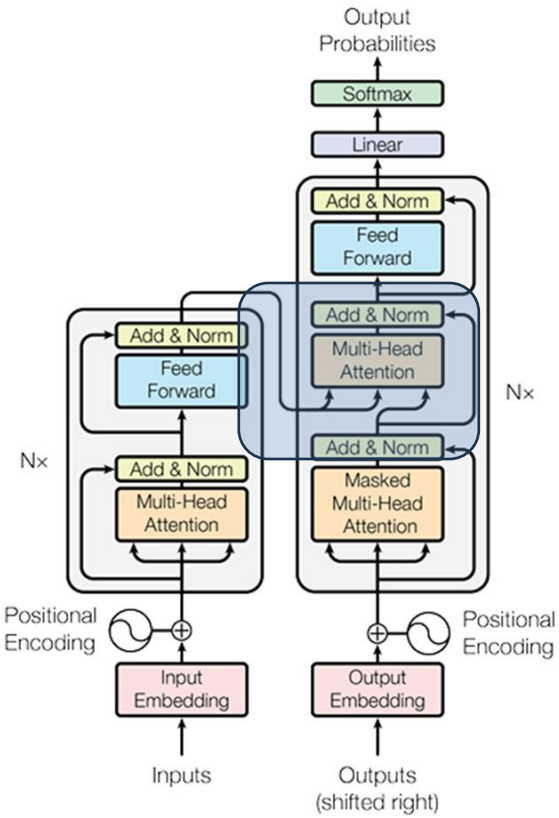
- In other words, the attention matrix will be of shape $n_{dec} \times n_{enc}$, which in this case is 3×5
  - Every token in the partial sequence of the decoder attends to all tokens of the encoder output.

Second Multi-Head Attention Step

- The weights indicate how much focus the decoder should give to the input token when generating new predictions.

Decoder Token	Attention Scores
The	[0.1, 0.45, 0.25, 0.1, 0.1]
cats	[0.2, 0.2, 0.5, 0.05, 0.05]
like	[0.1, 0.1, 0.1, 0.6, 0.1]
chasing	[0.03, 0.17, 0.05, 0.1, 0.55]

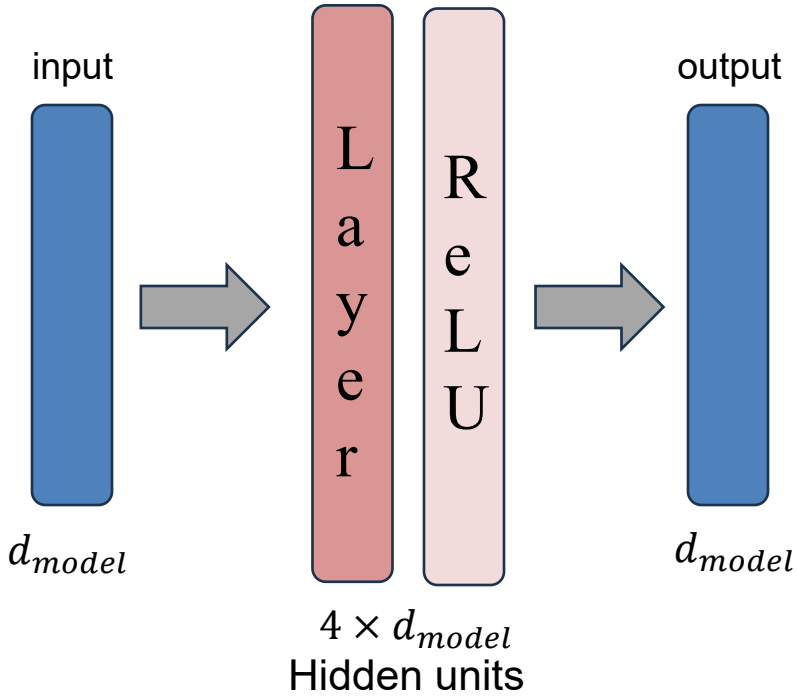
- As in the encoder block, these scores are used to weight the values V and then added to the output of the previous masked self-attention layer to produce new representations for each decoder token enriched with context from the input.

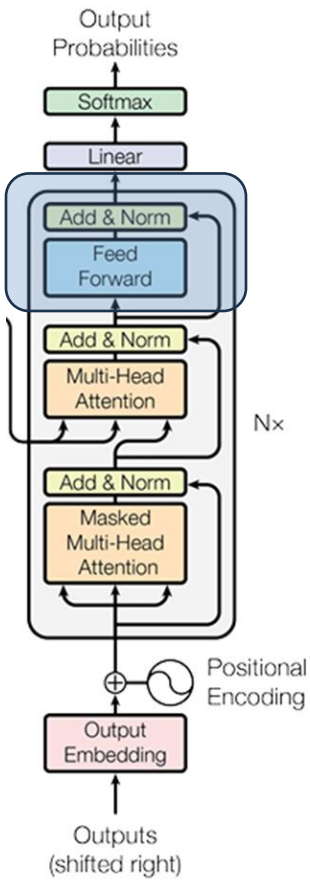


FFN Step

- After the two attention blocks (masked self-attention and cross-attention) in the transformer decoder, the next step is the feed-forward neural network (FFN). As before, this feed-forward layer is crucial for further processing the attention output, adding non-linearity, and helping the model learn complex transformations.
- Input to the FNN is the output from cross-attention.

Typical architecture:





Linear Projection

- After repeating previous steps N times (corresponding to the number of decoder blocks), the final step before we can get token predictions using our decoder embeddings is the linear projection.
- Input size: $n_{dec} \times d_{model}$, output size: $n_{dec} \times |V|$ (during training)
- This is done with a standard matrix multiplication, using another learned matrix sometimes called the "**unembedding**" matrix, with shape $d_{model} \times |V|$. The context vectors output by the decoder blocks are multiplied by this unembedding matrix to get **logits**:
  - $logits_i = context\ vector_i \cdot W_{unembed}$
  - where these logits are the **unnormalized score for probability** of each word in the dictionary for the corresponding token.
  - We use **softmax** on these logits to get a probability distribution over all words.

During Training

- During training, the linear layer is applied to each context vector from the decoder output independently. For example, apply it to “The”, then apply it to “cats”, and so on
- As mentioned in the previous slide, the output in this case would be a matrix of shape $n_{dec} \times$ vocab_size, and each row in this matrix would correspond to the logits of the next-word prediction for that token.
- Perform the multiply-add for each context vector, then apply softmax to turn each row into a probability distribution.
- Crossentropy loss is applied for each token, with the expected next token. For example:

“cats” | Unembedding matrix | logits (transposed) | Next word probabilities

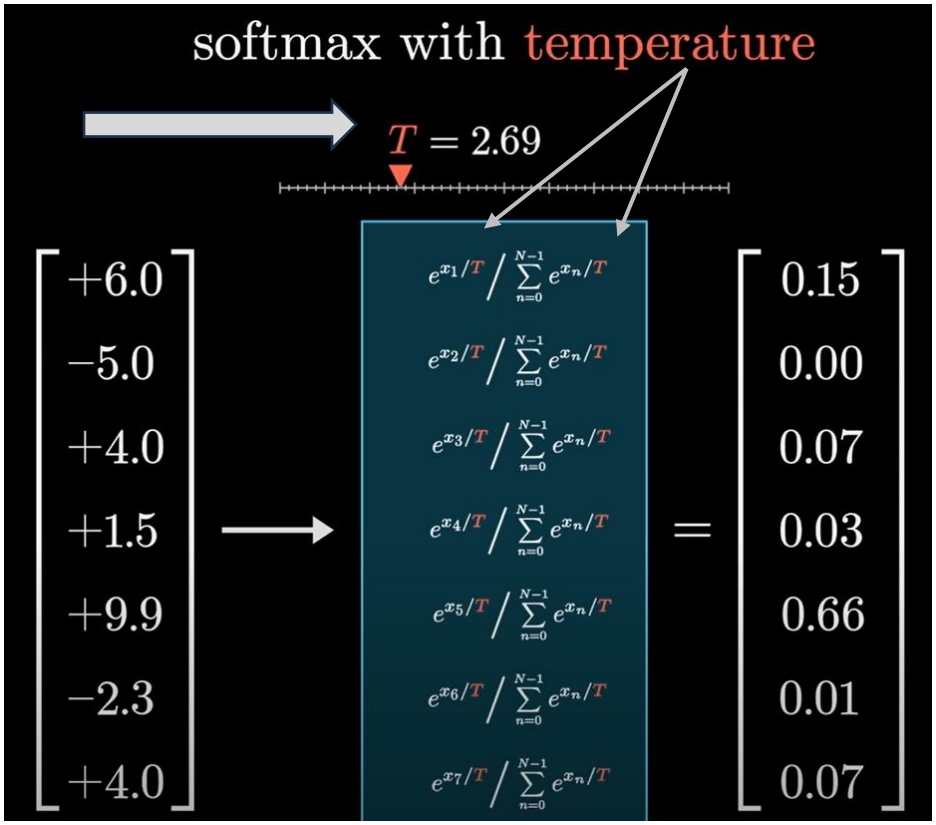
$$\underset{1 \times d_{model}}{[\quad \ldots \quad]} * \underset{d_{model} \times |V|}{\left[\quad\quad\right]} = \underset{(1 \times |V|)^T}{\begin{bmatrix} 0.9 \\ 2.8 \\ \vdots \\ 0.1 \\ 0.03 \end{bmatrix}} \xrightarrow{\textbf{Softmax}} \underset{(1 \times |V|)^T}{\begin{bmatrix} 0.3 \\ 0.6 \\ \vdots \\ 0.02 \\ 0.01 \end{bmatrix}} \begin{matrix} \text{ate} \\ \text{like} \\ \\ \text{the} \\ \text{zoo} \end{matrix}$$

During Inference

- During inference (for instance, when we want to generate next-word predictions), we only apply the linear layer to the **last context vector** of the decoder output.
  - After applying softmax, this gives us a single vector which is a probability distribution over the entire vocabulary, which we can sample from to get the next token in the sequence.
- Once a token is generated, we add it to the sequence and feed it back into the decoder to generate the next word.
  - This is an **autoregressive** method of generating words.

Temperature

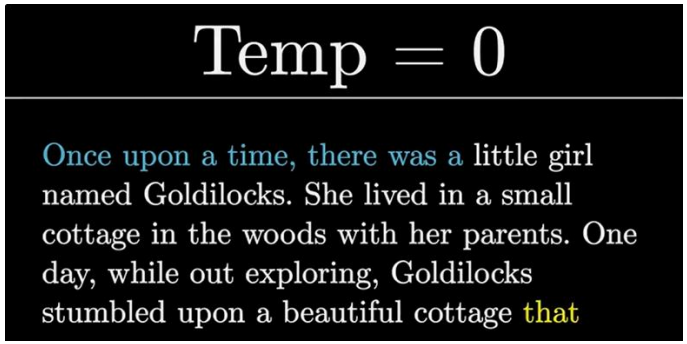
- This gives us a way to tweak the “randomness” of the model.
  - A higher temperature means more weight to the smaller values.
  - A lower temperature would mean bigger values dominate.
  - If T = 1e-10, then only the largest value would be 1 and all others 0 (greedy decoding or taking $argmax$).

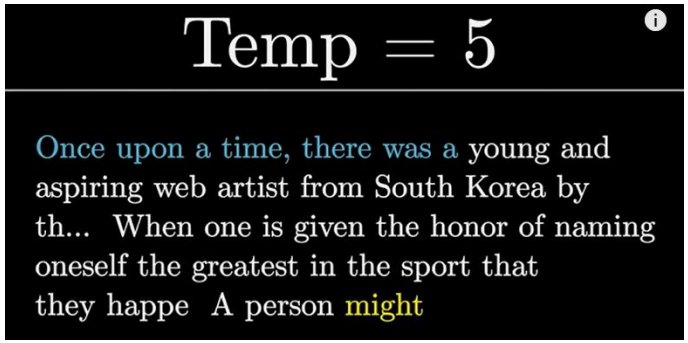


Temperature

Here's an example comparing the effect of temperature on next-word prediction, if we use "Once upon a time, there was a" as the seed text:

- Setting temperature to 0, we always pick the most predictable word for the next one in the sequence.
- However, setting temperature = 5, we see more "creativity" at first, but it quickly turns into non-sense





The Encoder-Decoder Transformer

Overall, a Transformer is a stack of Transformer blocks:

1. Including any number of Encoder blocks which process the input embeddings into context-aware representations.
2. And any number of Decoder blocks which generate output sequences based on the encoder outputs and previously generated tokens.

- In the Vaswani et. al paper, they used 6 Encoder blocks and 6 Decoder blocks.

ChatGPT

- ChatGPT only uses a decoder to generate natural language responses. A decoder-only architecture is sufficient because it:
  - Can handle causal attention, meaning it only looks at previous tokens, not future tokens.
  - Generates tokens one by one, allowing for natural language generation, since each token only depends on all the tokens before it.
- In Slide 9, while conceptually true, during actual training, the Transformer does not run Iteration 1, wait, run Iteration 2, wait, etc. It uses causal mask and computes all 5 of the rows simultaneously in one giant matrix multiplication.

Key Differences Between Encoder-Decoder and Decoder-Only Architectures:

Feature	**Encoder-Decoder**	**Decoder-Only**
Use Case	Translation, Summarization	Text Generation, Autocomplete
Architecture	Separate Encoder and Decoder	Only Decoder Blocks
Attention Type	Bidirectional (Encoder), Causal (Decoder)	Causal (Unidirectional)
Training Objective	Masked Language Modeling (MLM)	Autoregressive Language Modeling (LM)
Examples	T5, BART	GPT-3, GPT-4, GPT-2

MLM vs. Autoregressive LM

- Masked Language Modeling (MLM): "Fill-in-the-Blank"
  - **How it works:** We hide a word in the middle of the sentence and ask the model to guess it.
  - **What the model sees:** "The cats [MASK] chasing mice.“
  - **The Context:** Bidirectional attention. The model looks at the words before the blank ("The cats") AND the words after the blank ("chasing mice") at the exact same time.
  - **The Prediction:** It uses all that surrounding context to predict "like".
- Autoregressive Language Modeling (LM): *"Next-Word Prediction"*
  - **How it works:** We give the model a partial sentence and ask it to guess what comes next.
  - **What the model sees:** "The cats like _____"
  - **The Context:** Causal (Unidirectional attention). The model is forced to look *only* at the past words ("The cats like").
  - **The Prediction:** It calculates the probabilities based on that limited past context to guess that the next word is "chasing".

Different Models

Model	Number of Transforms block	Size of Embedding	Heads	Params	Size of the Dataset used in training	Training
Transformer-Base (original paper)	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	25	1.5B	40 GB	
GPT-3	96	12,288	96	175B	694GB	Unknown number of V100 GPUs (weeks to months)
GPT-4 (no official data)	120	12,288	96	1.76 T	1 petabyte (PB) 1000 TB	25,000 A100 GPUs
GPT-5	No Data !					

Simple Transformer Code Example

```python
import torch
import torch.nn as nn
import torch.optim as optim
import math

# Define a simple autoregressive chatbot transformer model
class SimpleChatbotTransformer(nn.Module):
    def __init__(self, vocab_size, d_model=64, nhead=4, num_layers=2, dim_feedforward=256, max_seq_length=20):
        super(SimpleChatbotTransformer, self).__init__()

        # Embedding and positional encoding
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.positional_encoding = nn.Parameter(self.get_positional_encoding(max_seq_length, d_model), requires_grad=False)

        # Transformer model
        self.transformer = nn.Transformer(d_model=d_model, nhead=nhead,
                                          num_encoder_layers=num_layers,
                                          num_decoder_layers=num_layers,
                                          dim_feedforward=dim_feedforward)

        # Output layer to convert transformer outputs to vocabulary logits
        self.fc_out = nn.Linear(d_model, vocab_size)

    def get_positional_encoding(self, max_seq_length, d_model):
        # Generate fixed positional encoding
        positional_encoding = torch.zeros(max_seq_length, d_model)
        position = torch.arange(0, max_seq_length, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * -(math.log(10000.0) / d_model))
        positional_encoding[:, 0::2] = torch.sin(position * div_term)
        positional_encoding[:, 1::2] = torch.cos(position * div_term)
        return positional_encoding.unsqueeze(1)
```

Simple Transformer Code Example

```
 def forward(self, src, tgt, tgt_mask=None):
        # Add fixed positional encoding to the embeddings
        src_emb = self.embedding(src) + self.positional_encoding[:src.size(0), :]
        tgt_emb = self.embedding(tgt) + self.positional_encoding[:tgt.size(0), :]

        # Pass through the transformer model
        transformer_output = self.transformer(src_emb, tgt_emb, tgt_mask=tgt_mask)

        # Project the output to the vocabulary size
        output = self.fc_out(transformer_output)
        return output

# Define a simple vocabulary and conversation pairs
vocab = {
    "<PAD>": 0, "<START>": 1, "<END>": 2,
    "Hello": 3, "how": 4, "are": 5, "you": 6, "?": 7,
    "I": 8, "am": 9, "fine": 10, "thank": 11, "!": 12,
    "what": 13, "is": 14, "your": 15, "name": 16,
    "my": 17, "Chatbot": 18
}

# Example conversation pairs (input → output)
# In real life you will have millions of these
conversations = [
    (["Hello", "how", "are", "you", "?"], ["<START>", "I", "am", "fine", "thank", "you", "!","<END>"]),
    (["what", "is", "your", "name", "?"], ["<START>", "my", "name", "is", "Chatbot", "!","<END>"])
]

# Convert conversation pairs to tensor indices
def tokenize_sentence(sentence, vocab):
    return [vocab[word] for word in sentence]
```

Simple Transformer Code Example

```python
# Training setup
model = SimpleChatbotTransformer(len(vocab))
criterion = nn.CrossEntropyLoss(ignore_index=0)  # Ignore padding index
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
num_epochs = 20
for epoch in range(num_epochs):
    model.train()
    total_loss = 0

    for src_sentence, tgt_sentence in conversations:
        optimizer.zero_grad()

        # Prepare input and target sequences
        src_indices = torch.tensor([tokenize_sentence(src_sentence, vocab)]).T
        tgt_indices = torch.tensor([tokenize_sentence(tgt_sentence, vocab)]).T

        # Generate a mask for causal decoding with the correct shape
        tgt_mask = nn.Transformer.generate_square_subsequent_mask(tgt_indices[:-1].size(0)).to(tgt_indices.device)

        # Forward pass
        output = model(src_indices, tgt_indices[:-1], tgt_mask=tgt_mask)

        # Compute loss
        output = output.view(-1, len(vocab))
        target = tgt_indices[1:].view(-1)
        loss = criterion(output, target)

        # Backpropagation
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {total_loss:.4f}")
```

Simple Transformer Code Example

```python
# Evaluation: Autoregressive inference with max length 15
model.eval()

def generate_response(input_text, max_length=15):
    # Convert the input text to source indices
    src_indices = torch.tensor([[vocab[word] for word in input_text.split()]]).T

    # Initialize the generated tokens with the <START> token
    generated_tokens = ["<START>"]

    for _ in range(max_length):
        # Convert generated tokens to tensor indices
        tgt_indices = torch.tensor([[vocab[token] for token in generated_tokens]]).T

        # Generate a causal mask for the target sequence
        tgt_mask = nn.Transformer.generate_square_subsequent_mask(tgt_indices.size(0)).to(tgt_indices.device)

        with torch.no_grad():
            # Forward pass through the model
            output = model(src_indices, tgt_indices, tgt_mask=tgt_mask)

        # Select the most probable token (greedy decoding)
        next_token_logits = output[-1, 0, :]
        predicted_token_index = next_token_logits.argmax().item()

        # Convert index to token
        predicted_token = list(vocab.keys())[list(vocab.values()).index(predicted_token_index)]

        # Stop if the <END> token is generated
        if predicted_token == "<END>":
            break

        # Append the predicted token to the list
        generated_tokens.append(predicted_token)

    # Return the generated response as a string (excluding the <START> token)
    return " ".join(generated_tokens[1:])

# Test the chatbot
print("User: Hello how are you ?")
print("Chatbot:", generate_response("Hello how are you ?"))

print("User: what is your name ?")
print("Chatbot:", generate_response("what is your name ?"))
```

Vision Transformers

What is the Goal of Vision Transfomers?

- We want to use the attention mechanism of the Transformer architecture on **images** to be able to perform computer vision tasks like image classification.
- The intuition behind why this could work can be seen in the way that we can represent images as vectors, just like we have been doing for words in this lecture. Recall from earlier lectures that we can extract "feature vectors" using CNNs that represent important information about an image!
- Similarly, we can use Encoder blocks to extract vectors from an image. The process would look like:

Transformer Encoder → Softmax Classifier →

Cat
Dog
Horse
Ship
Plane
…

Early Research

ICML 2020

Generative Pretraining from Pixels

Mark Chen [1] Alec Radford [1] Rewon Child [1] Jeff Wu [1] Heewoo Jun [1] David Luan [1] Ilya Sutskever [1]

[1]OpenAI, San Francisco, CA, USA. Correspondence to: Mark Chen <mark@openai.com>.

Proceedings of the 37th International Conference on Machine Learning, Vienna, Austria, PMLR 119, 2020. Copyright 2020 by the author(s).

Cited by 1905

Early Research

- The paper introduces Image-GPT (iGPT). Here are the main takeaways from it:
  - An image consists of pixels
  - Treat each pixel like a token
    - Each color is represented using 24 bits per channel
    - Which means values can be between 0-255 per color
    - This means our vocab size is 16,777,216 since this is the number of unique colors that each pixel can have!
  - Possible solution:
    - Use 8 bits, which would make vocab size = 8x8x8 = 512

Proposed Model of I-GPT

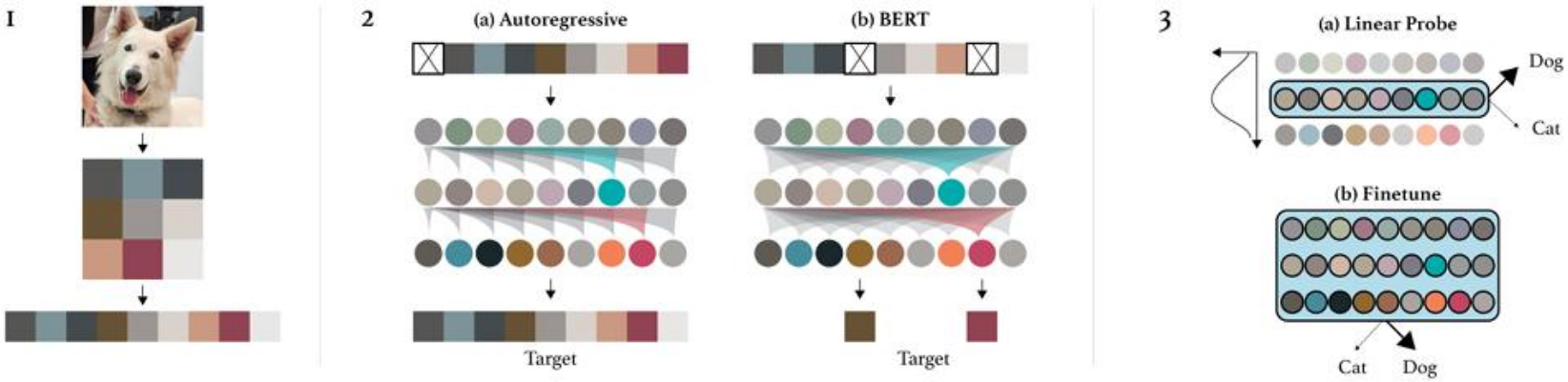


Figure 1. An overview of our approach. First, we pre-process raw images by resizing to a low resolution and reshaping into a 1D sequence. We then chose one of two pre-training objectives, auto-regressive next pixel prediction or masked pixel prediction. Finally, we evaluate the representations learned by these objectives with linear probes or fine-tuning.

- For an $R \times R$ image, the self-attention mechanism would compare all R^2 tokens against themselves
  - This would make the attention matrix on the order of R^4!
  - The computational and memory cost would be too high.

Limitations of I-GPT

- We train iGPT-S, iGPT-M, and iGPT-L, transformers containing 76M, 455M, and 1.4B parameters respectively, on ImageNet. We also train iGPT-XLD, a 6.8 billion parameter transformer, on a mix of ImageNet and images from the web. Due to the large computational cost of modeling long sequences with dense attention, we train at the low resolutions of 32x32, 48x48, and 64x64.
- Because we use the generic sequence transformer used for GPT-2 in language, our method requires large amounts of compute: iGPT-L was trained for roughly 2500 V100-days
- Generative models can exhibit biases that are a consequence of the data they've been trained on. Many of these biases are useful, like assuming that a combination of brown and green pixels represents a branch covered in leaves, then using this bias to continue the image. But some of these biases will be harmful, when considered through a lens of fairness and representation. For instance, if the model develops a visual notion of a scientist that skews male, then it might consistently complete images of scientists with male-presenting people, rather than a mix of genders.

A Different Approach

A **Linear Probe** is a simple classification model (usually a single-layer linear classifier) that is trained on **frozen feature representations** extracted from a **pretrained model**. It is commonly used in **self-supervised learning (SSL) and transfer learning** to evaluate the quality of learned representations.

1. Pretrain a model in an unsupervised way
2. Extract feature representations from an intermediate layer (often from the last convolutional layer or a transformer encoder).
3. Train a simple linear classifier (logistic regression or softmax layer) on top using labeled data.
4. Evaluate classification accuracy

Table 1. Comparing linear probe accuracies between our models and state-of-the-art models utilizing unsupervised ImageNet transfer or supervised ImageNet transfer.

Model	Acc	Unsup Transfer	Sup Transfer
CIFAR-10			
ResNet-152	94		√
SimCLR	95.3	√	
iGPT-L	96.3	√	
CIFAR-100			
ResNet-152	78.0		√
SimCLR	80.2	√	
iGPT-L	82.8	√	
STL-10			
AMDIM-L	94.2	√	
iGPT-L	95.5	√	

Unsupervised Transfer Learning

Unsup Transfer in the table refers to **Unsupervised Transfer Learning**, which means the model was **pretrained on an unsupervised learning task** before being fine-tuned for a downstream task like CIFAR-10, CIFAR-100, or STL-10 classification.

In SimCLR, a model (e.g., ResNet-50) is pretrained without labels using contrastive learning. The goal is to learn representations that group similar images together in an embedding space.

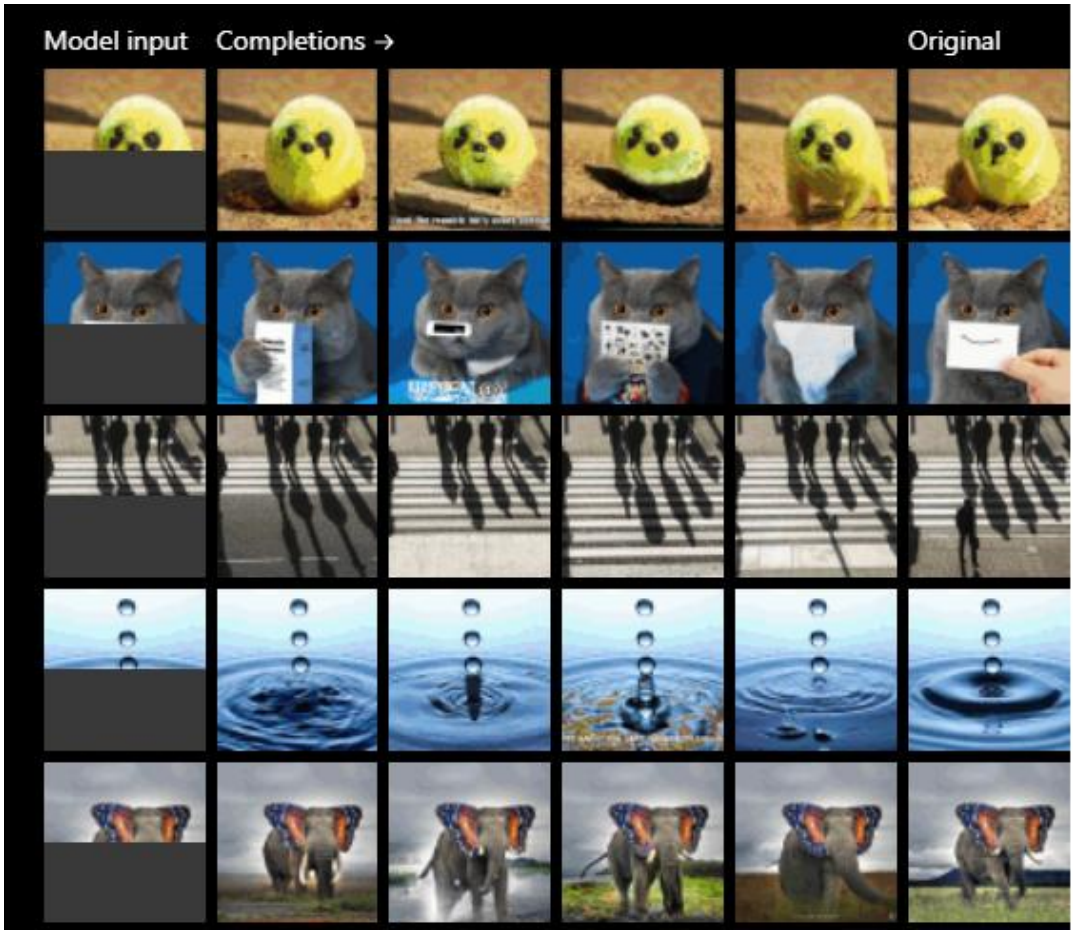
1. Data Augmentation:
- Take an image and create two random augmented versions of it (e.g., cropping, color distortion, blurring).
- These augmented images are called "positive pairs.“

2. Feature Extraction:
- A CNN (e.g., ResNet-50) extracts feature representations for both augmented images.

3. Contrastive Loss :
- The model learns to pull positive pairs close together in embedding space.
- It simultaneously pushes apart images from different instances (negative samples).

4. Pretrained Model Output:
- The model now has a representation space where semantically similar images are close together, even without labels.

Image Completions



Model-generated completions of human-provided half-images. We sample the remaining halves with temperature 1 and without tricks like beam search or nucleus sampling. While we showcase our favorite completions in the first panel, we do not cherry-pick images or completions in all following panels.

Vision Transformers Paper

ICLR 2021

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Alexey Dosovitskiy*,†, Lucas Beyer*, Alexander Kolesnikov*, Dirk Weissenborn*, Xiaohua Zhai*, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby*,†

*equal technical contribution, †equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulsby}@google.com

Cited by 56,233

Vision Transformer Architecture

- The Vision Transformer is an Encoder-only architecture, very similar to what we have already seen! Except instead of pixels, it uses **image patches** as tokens.

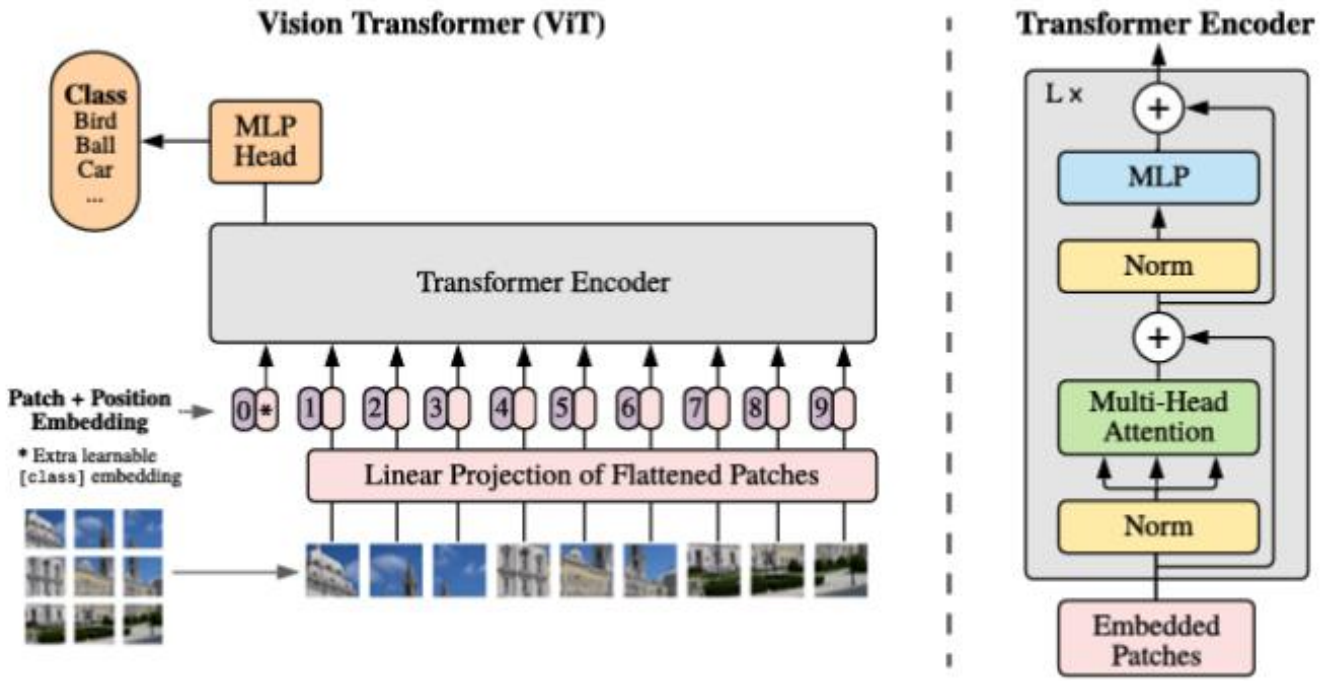
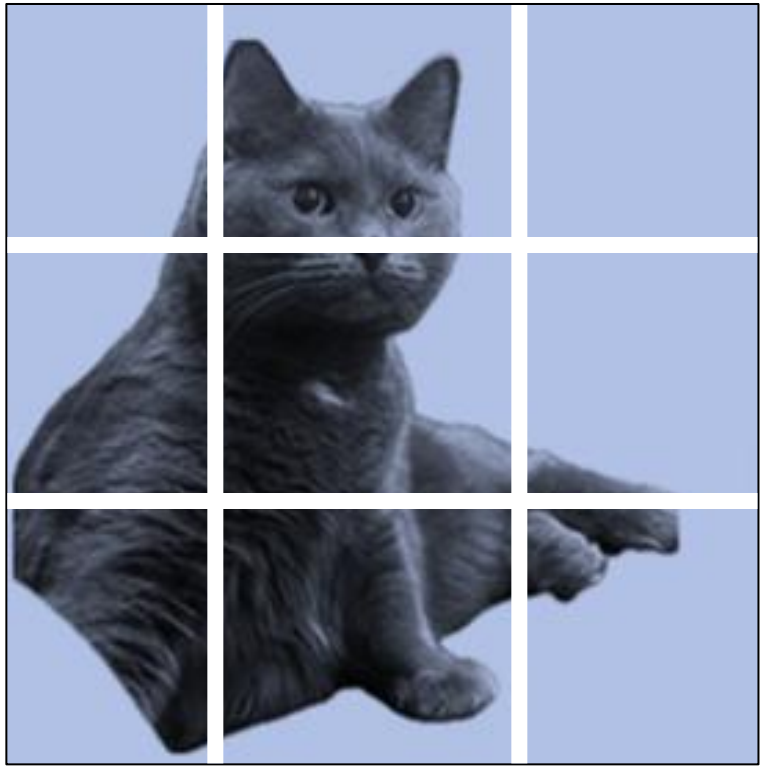


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

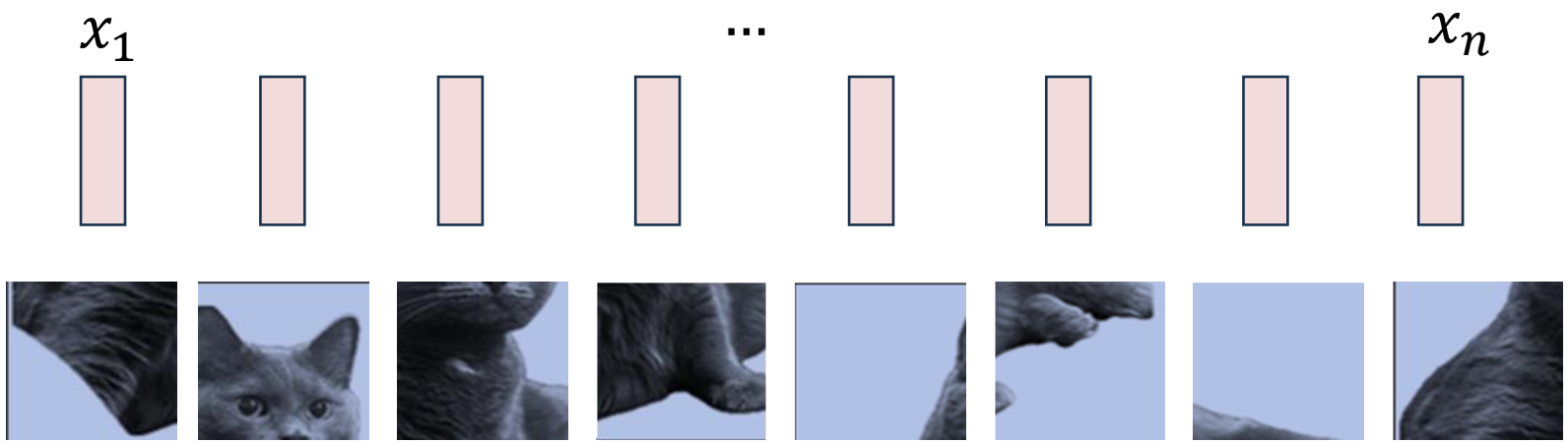
Steps for a ViT Encoder

- **<u>Step 1</u>**: divide the image into $d_1 \times d_2 \times c$ patches
  - Patches may overlap, but were not overlapping in original paper
  - Patch size = 16x16
  - Stride = 16x16 (i.e. no overlap)



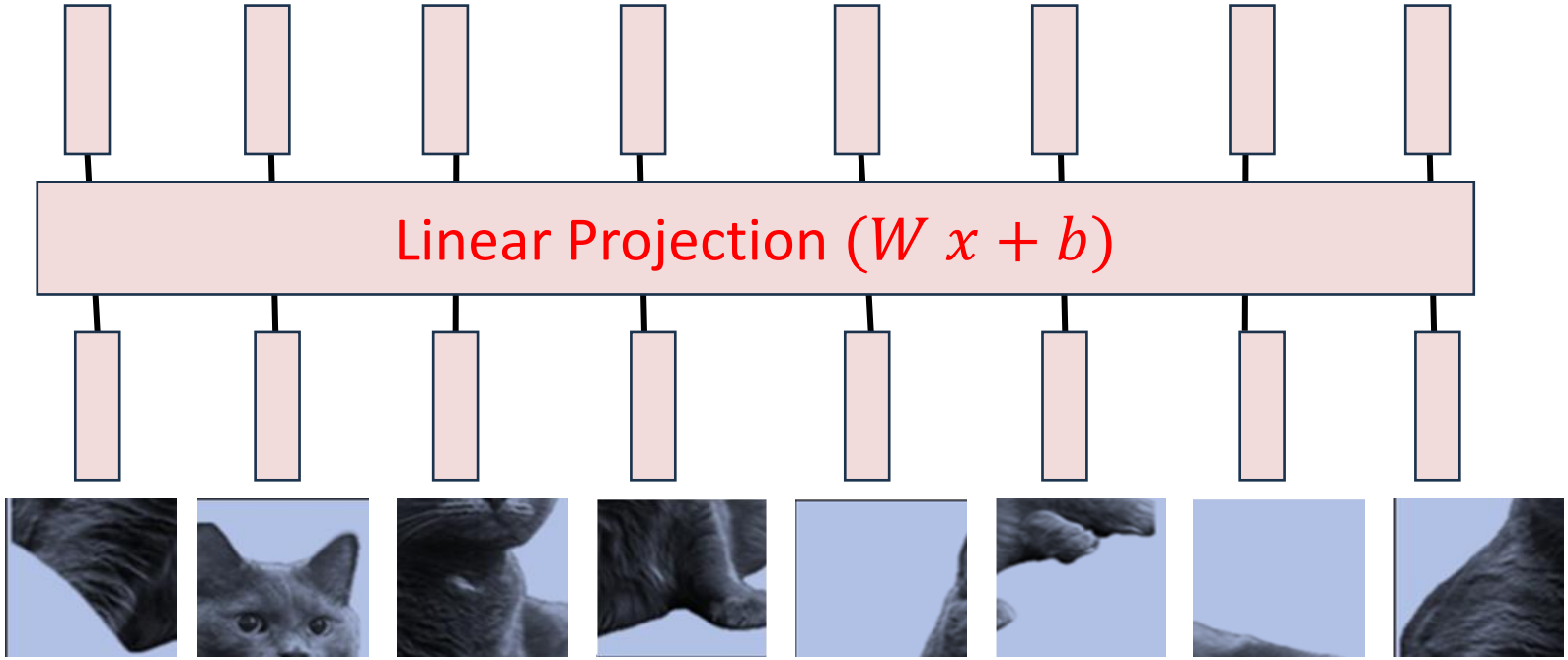
Steps for a ViT Encoder

- **<u>Step 2</u>**: flatten each patch into a 1-dimensional vector; i.e. if they were $d_1 \times d_2 \times c$ before, turn them into $1 \times d_1 d_2 c$



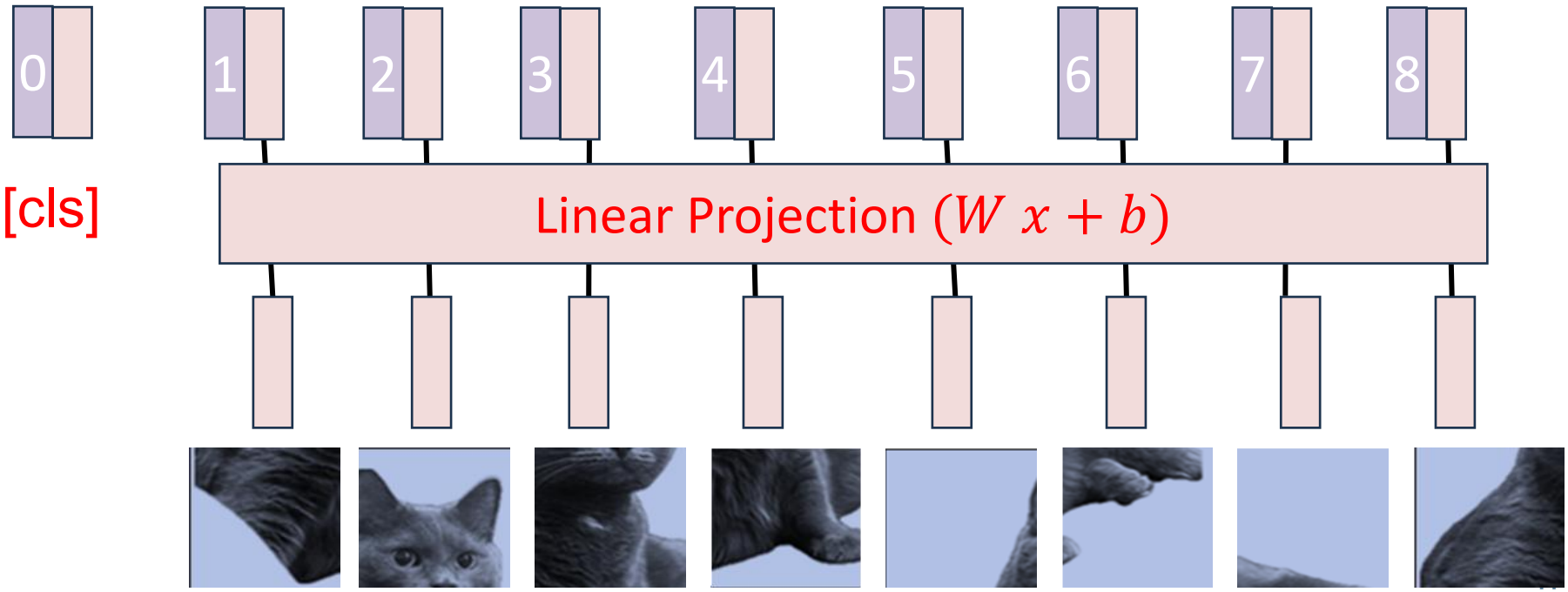
Steps for a ViT Encoder

- **<u>Step 3</u>**: Apply linear projection
  - This serves to reduce dimensionality
  - Guarantees a fixed size output
  - Better complex feature representations for images, rather than working with raw pixels
  - W and b are learnable, and resulting embeddings will be used as input to the transformer.



Steps for a ViT Encoder

- **<u>Step 4</u>**: add positional encoding
  - Similar to the case of the Encoder we saw earlier, we need to solve the problem of permutation equivariance.
  - We also need to add the “cls” token, which will be used for classification. We give it position 0.
  - In the paper, they used the same fixed sinusoidal positional embeddings.

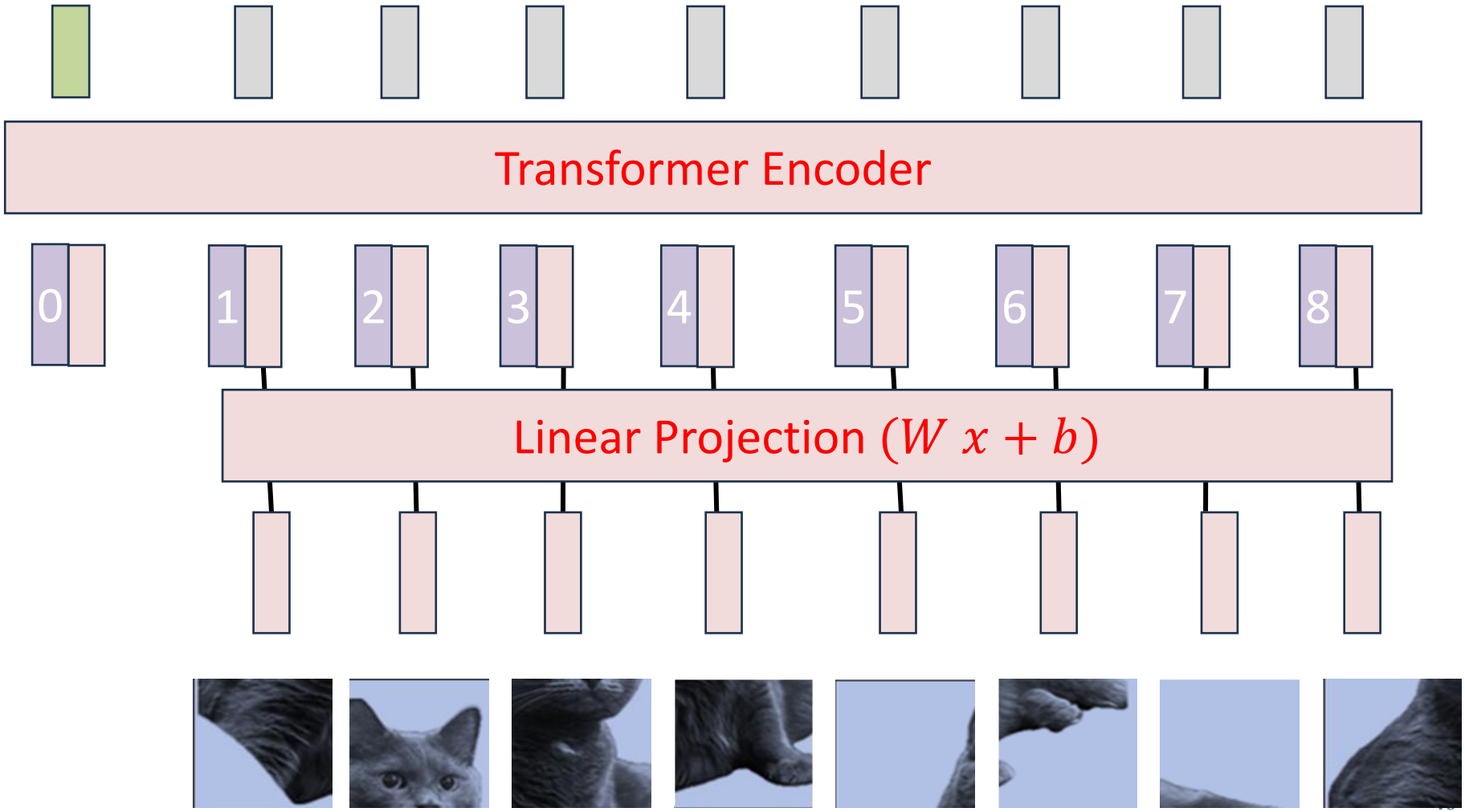


CLS Token

- This is an extra token (for classification) and has a learnable embedding, which is added to the input sequence at position 0.
  - It is inspired by BERT, used for sentence-level tasks.
  - Its embeddings are initialized randomly and updated in training.
  - After the Transformer encoder layer, **only the CLS token** is passed to the MLP classification head.
- Why do we need a CLS token?
  - It learns to capture the overall context of the image
  - It reduces complexity of the overall classification process since we only have to deal with one context vector instead of all of them.
- In more advanced transformers (Swin), the CLS token is replaced by a Global Average Pool over all context vectors in the transformer output.

Steps for a ViT Encoder

- **<u>Step 5</u>**: pass embeddings through a Transformer Encoder
  - Same as before, could be as many Encoder blocks as you like.
  - The output vectors are all ignored except the CLS output.



Steps for a ViT Encoder

- **<u>Step 6</u>**: pass the embeddings of the CLS token into the MLP
  - Ignore the other context vectors

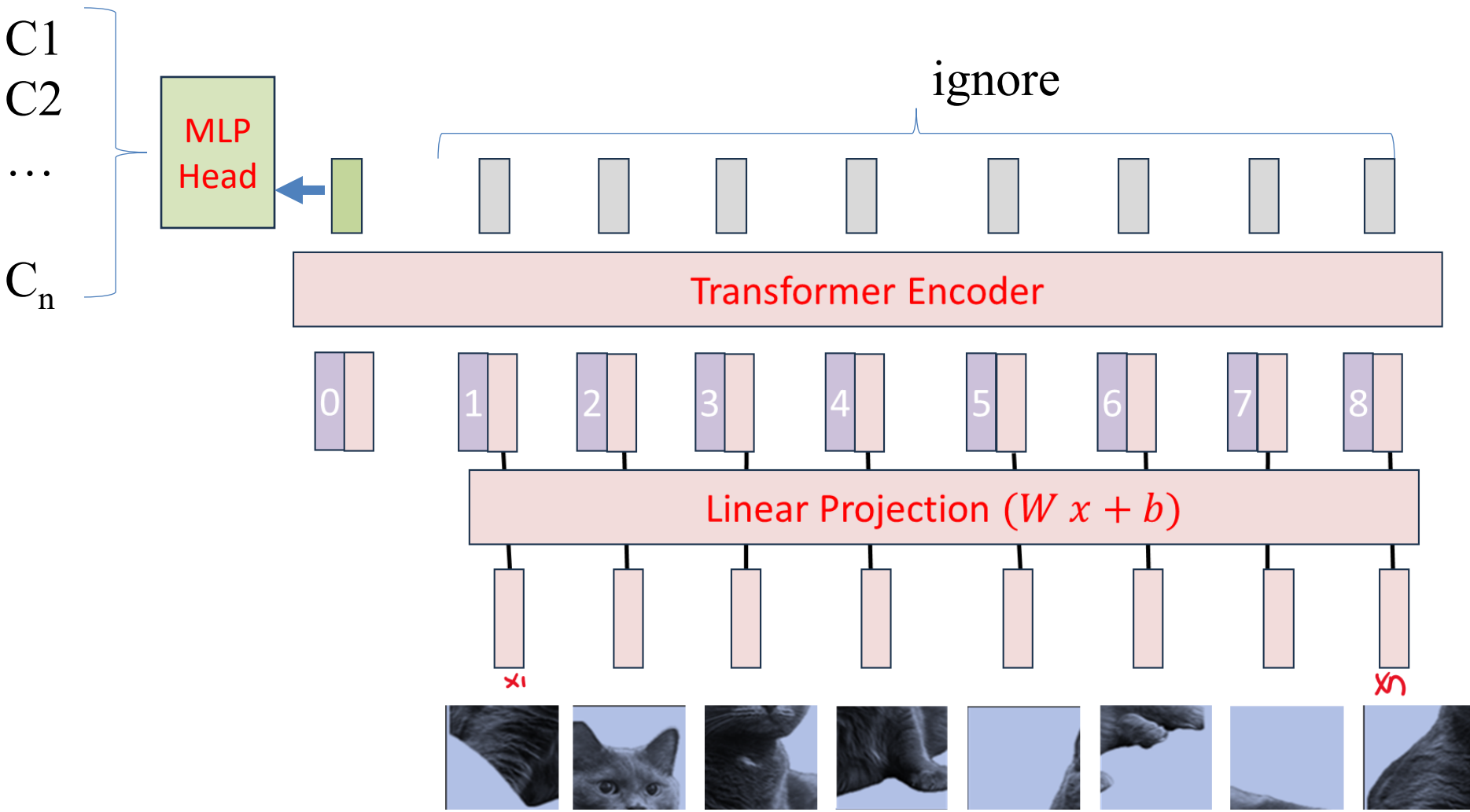


Table of Parameters

- Here is the number of parameters per model, ranging from Base to Huge:

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

  - Layers represents the number of encoder blocks
  - Hidden size is the dimension of the patch embeddings (controlled through the linear projection layer)
  - MLP size refers to the size of the MLP **inside** the encoder (i.e. the number of neurons in the hidden layer, e.g. $4 \times d_{model}$

Calculating the Number of Params

Image is 224 x 224, so each patch = $16 \times 16 \times 3 = 768$

- Linear projection: $\mathrm{W}_{\mathrm{patch}} = (768 \times 768) + 768 = 590{,}592$
- Transformer block: Q, K, V matrices
  - $W_{QKV} = 3 \times (768 \times 768) = 1{,}769{,}472$
  - $W_{TB-Linear} = 768 \times 768 = 589{,}824$
  - For one head: $1{,}769{,}472 + 589{,}829 = 2{,}359{,}296$
  - For twelve heads: $28{,}311{,}552$
- MLP:
  - First hidden layer: $768 \times 3072 = 2{,}359{,}296$
  - Second hidden layer: $3072 \times 768 = 2{,}359{,}296$
  - For one head: $2{,}359{,}296 + 2{,}359{,}296 = 4{,}718{,}592$
  - For twelve heads: $56{,}623{,}104$
- Classification MLP (for 1000 classes): $768 \times 1000 = 768{,}000$

Calculating the Number of Params

Component	Parameters
Patch Embedding	590,592
Transformer Encoders (MSA)	28,311,552
Transformer Encoders (MLP)	56,623,104
Final Classification MLP	768,000
Total	**86,293,284 (~86M)**

Excerpt from the Paper

- This excerpt mentions the variants of the ViT models, as well as mentioning a modified CNN architecture they named “Resnet BiT” which has improved transfer learning performance.

Model Variants. We base ViT configurations on those used for BERT (Devlin et al., 2019), as summarized in Table 1. The “Base” and “Large” models are directly adopted from BERT and we add the larger “Huge” model. In what follows we use brief notation to indicate the model size and the input patch size: for instance, ViT-L/16 means the “Large” variant with 16×16 input patch size. Note that the Transformer’s sequence length is inversely proportional to the square of the patch size, thus models with smaller patch size are computationally more expensive.

For the baseline CNNs, we use ResNet (He et al., 2016), but replace the Batch Normalization layers (Ioffe & Szegedy, 2015) with Group Normalization (Wu & He, 2018), and used standardized convolutions (Qiao et al., 2019). These modifications improve transfer (Kolesnikov et al., 2020), and we denote the modified model “ResNet (BiT)”.

Table of Performance

- These are some comparisons on the performance of their architecture on various benchmark datasets across the various sizes.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** $\pm$ 0.04	87.76 $\pm$ 0.03	85.30 $\pm$ 0.02	87.54 $\pm$ 0.02	88.4/88.5*
ImageNet ReaL	**90.72** $\pm$ 0.05	90.54 $\pm$ 0.03	88.62 $\pm$ 0.05	90.54	90.55
CIFAR-10	**99.50** $\pm$ 0.06	99.42 $\pm$ 0.03	99.15 $\pm$ 0.03	99.37 $\pm$ 0.06	–
CIFAR-100	**94.55** $\pm$ 0.04	93.90 $\pm$ 0.05	93.25 $\pm$ 0.05	93.51 $\pm$ 0.08	–
Oxford-IIIT Pets	**97.56** $\pm$ 0.03	97.32 $\pm$ 0.11	94.67 $\pm$ 0.15	96.62 $\pm$ 0.23	–
Oxford Flowers-102	99.68 $\pm$ 0.02	**99.74** $\pm$ 0.00	99.61 $\pm$ 0.02	99.63 $\pm$ 0.03	–
VTAB (19 tasks)	**77.63** $\pm$ 0.23	76.28 $\pm$ 0.46	72.72 $\pm$ 0.21	76.29 $\pm$ 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Table 2: Comparison with state of the art on popular image classification benchmarks. We report mean and standard deviation of the accuracies, averaged over three fine-tuning runs. Vision Transformer models pre-trained on the JFT-300M dataset outperform ResNet-based baselines on all datasets, while taking substantially less computational resources to pre-train. ViT pre-trained on the smaller public ImageNet-21k dataset performs well too. *Slightly improved 88.5% result reported in Touvron et al. (2020).

- Three sizes of ViT: B = Base (86M), L = Large (300M+), H = Huge (600M+ params)
- The numbers /14, /16, /32 indicate the **patch size**. Smaller patches = more computation needed per image

Performance Comparison - ImageNet

- ResNet outperforms the ViTs when pre-trained on ImageNet (~1.2M images)
- The smaller ViT-B also outperforms ViT-L

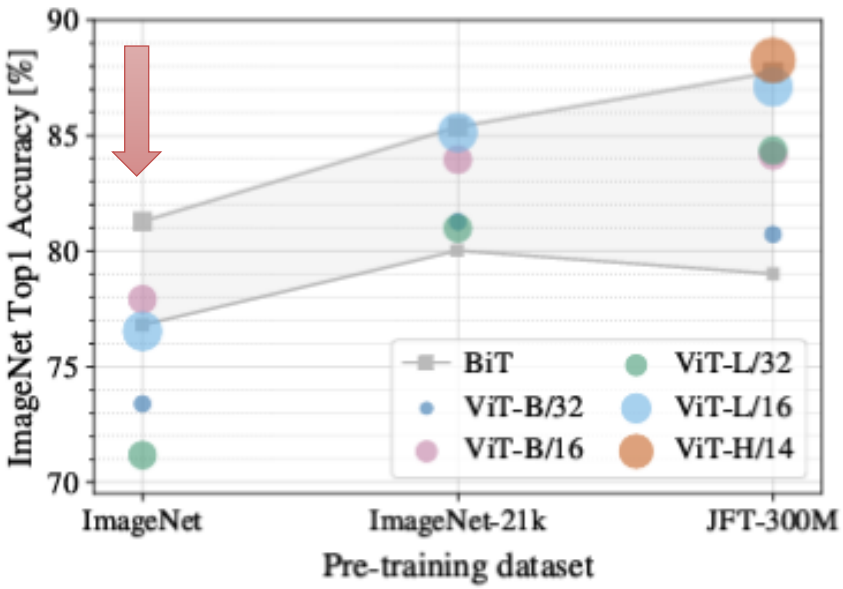


Figure 3: Transfer to ImageNet. While large ViT models perform worse than BiT ResNets (shaded area) when pre-trained on small datasets, they shine when pre-trained on larger datasets. Similarly, larger ViT variants overtake smaller ones as the dataset grows.

Performance Comparison – ImageNet 21K

- ViT-L is matches the ResNet with larger dataset (~14M images)
- ViT-L has overtaken ViT-B

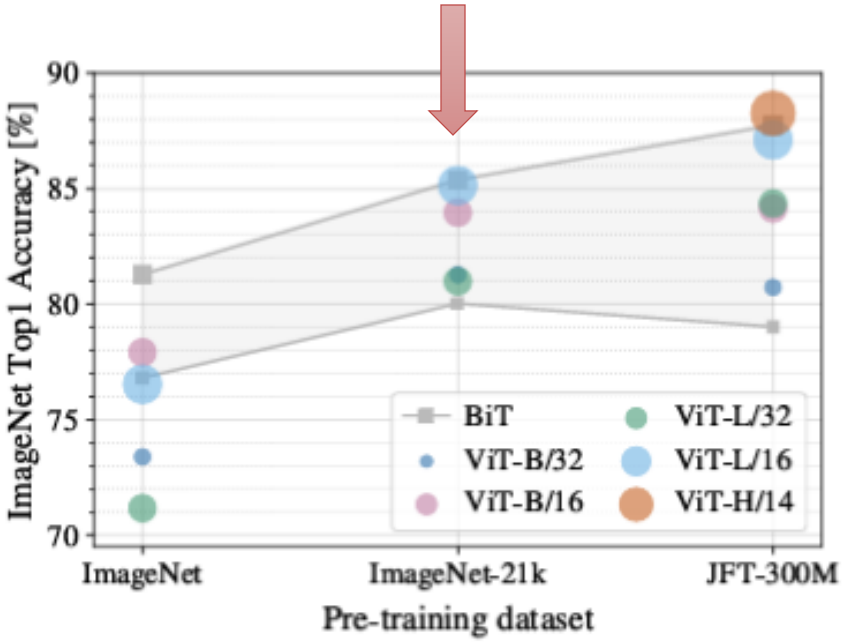


Figure 3: Transfer to ImageNet. While large ViT models perform worse than BiT ResNets (shaded area) when pre-trained on small datasets, they shine when pre-trained on larger datasets. Similarly, larger ViT variants overtake smaller ones as the dataset grows.

Performance Comparison – JFT 300M

- ViT-H is the best performing when pretrained on JFT (~300M images)
- Vision Transformer model overtakes ResNet

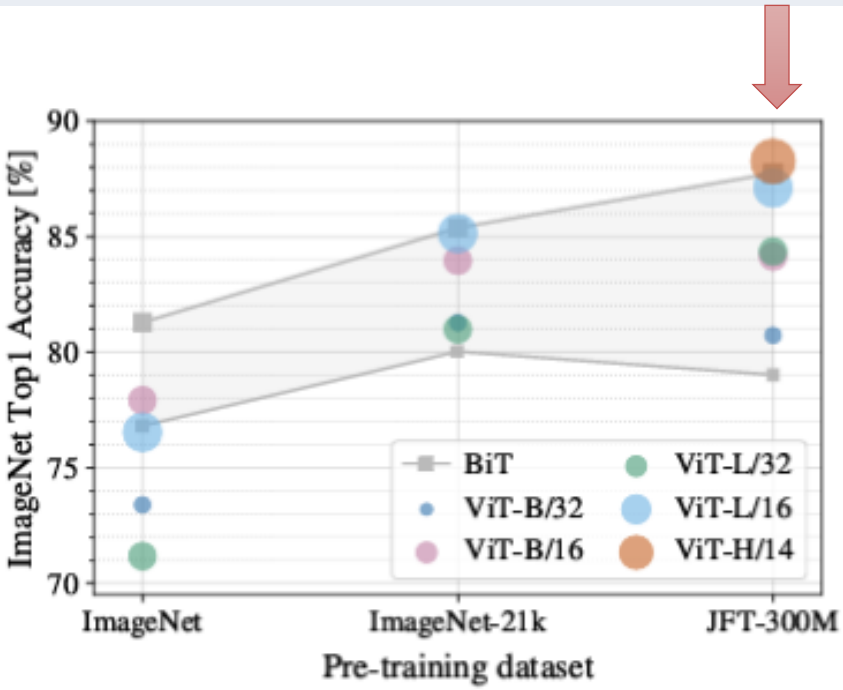


Figure 3: Transfer to ImageNet. While large ViT models perform worse than BiT ResNets (shaded area) when pre-trained on small datasets, they shine when pre-trained on larger datasets. Similarly, larger ViT variants overtake smaller ones as the dataset grows.

Performance vs Compute

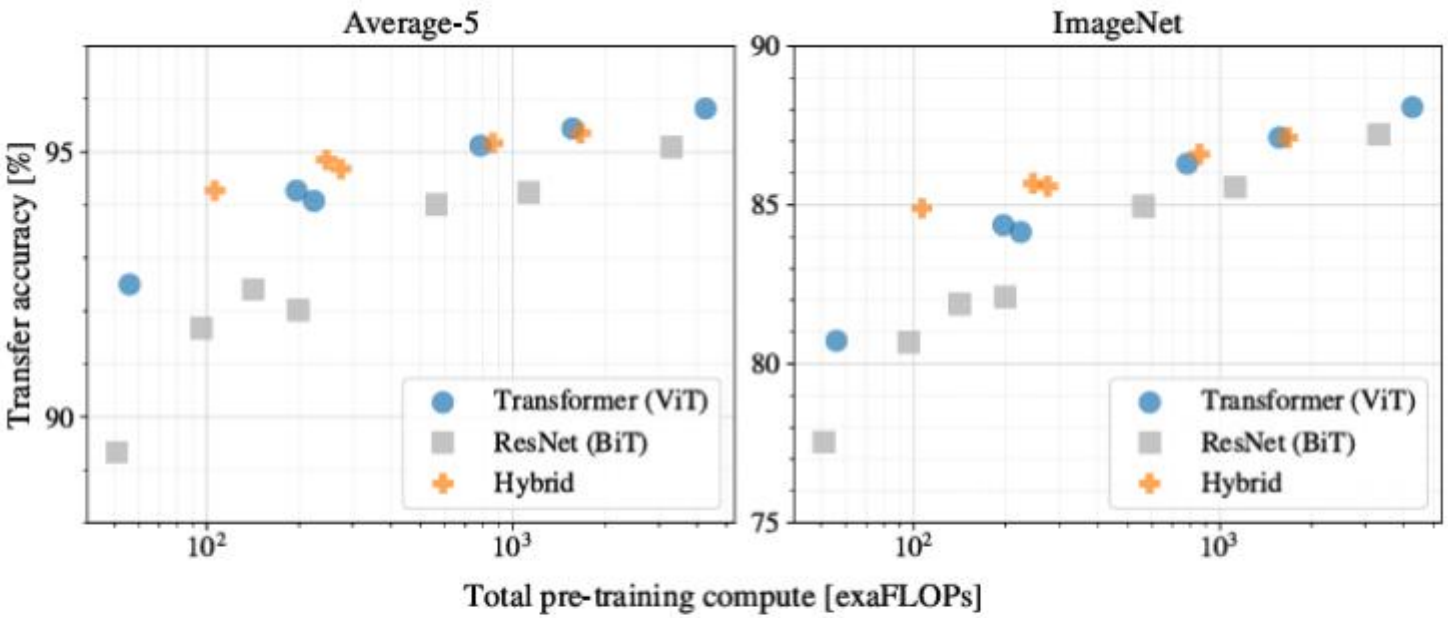


Figure 5: Performance versus pre-training compute for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

ViTs are consistently better than ResNet for the same amount of pre-training compute

"Average-5" refers to the average performance over the 5 datasets from Table 2.

Why ViT's Need Larger Datasets

Claim: ViT Models Have "Less Inductive Bias" Than ResNets, So They Need More Pretraining Data to Learn Good Features

This claim is fundamental to understanding why ViTs require large-scale datasets like JFT-300M for pretraining, whereas ResNet can learn efficiently from smaller datasets like ImageNet.

What is Inductive Bias?

- Inductive bias refers to built-in assumptions in a model that help it generalize better with limited data.
- A model with strong inductive bias can learn useful features more efficiently.
- A model with less inductive bias relies more on data-driven learning and requires larger datasets.
- Example: Human Learning Analogy
  - If a person already knows the general shape of a cat, they can recognize a new cat from just a few examples (strong inductive bias).If a person has no prior knowledge about what a cat looks like, they need to see many different images to learn the same (weak inductive bias).

Inductive Bias in CNNs

CNNs like ResNets have **strong inductive bias**, because of 2 key properties:

1) Locality (Spatial Hierarchy):
- CNNs focus on small nearby regions first before combining information.
- Early layers detect edges, textures, and patterns, while deeper layers detect complex structures (e.g., faces, objects).
- Result: Even with small datasets, CNNs learn useful features efficiently.

2) Translation Equivariance (Weight Sharing):
- Convolutions use shared kernels across the image, meaning a feature (like an edge) detected in one part of the image can be recognized elsewhere in the image.
- Result: CNNs generalize well across positions, without extensive data

Therefore, ResNets can learn robust image representations efficiently even from smaller datasets like ImageNet (~1.3M images).

Less Inductive Bias in ViTs

ViTs use self-attention which lacks the built-in spatial priors of CNNs.

1) No Locality (All Patches Treated Equally):

- Unlike CNNs, ViTs divide an image into non-overlapping patches and process them without explicitly capturing local spatial structure.
- The model must learn spatial relationships entirely from data.
- Result: ViTs require more data to understand basic structures like edges, textures, and shapes.

2) No Translation Equivariance:

- Self-attention does not share weights across different regions.
- A pattern detected in one part of an image is not automatically recognized elsewhere.
- Result: ViTs must learn positional relationships explicitly from data

Conclusion: ViTs need massive datasets (e.g., JFT-300M, ImageNet-21k) for pretraining before they can generalize effectively.

Visualizing Attention in ViTs

Left Column (Input): Shows the original images given to the Vision Transformer (ViT).
Right Column (Attention): Visualizes where the CLS token (classification token) is focusing when making a prediction.
Bright areas in attention maps: Indicate regions that contribute most to the model's decision.
Dark areas in attention maps: Represent regions ignored by the model.

Top image (Dog): The model mainly focuses on the dog's face, ignoring the grass background
Middle image (Airplane): The model strongly attends to the airplane, with less focus on the sky.
Bottom image (Snake): The model highlights the snake's body, disregarding the leaves and background.

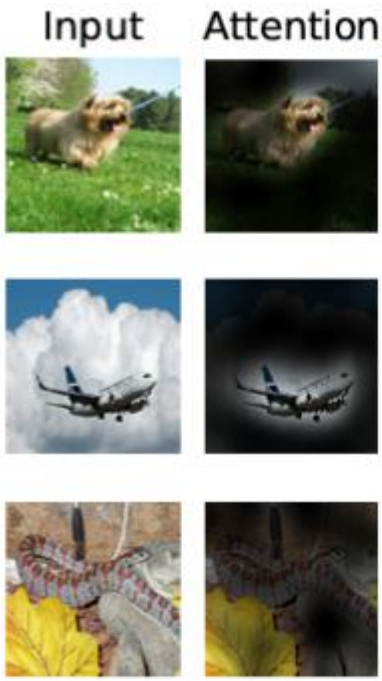


Figure 6: Representative examples of attention from the output token to the input space. See Appendix D.7 for details.

CLS Token